

Segment Tree

Enzo Vivallo

enzovivallo@estudiante.uc.cl



Problema

Se te entrega una lista de enteros a de tamaño n ($1 \leq n \leq 2 \cdot 10^5$), y cada elemento cumple $|a_i| \leq 10^9$. Nos piden responder q ($1 \leq q \leq 2 \cdot 10^5$) consultas del tipo:

¿Cuál es el mínimo en el rango $[l, r]$?



Solución naive

Para cada consulta, recorrer el rango $[l, r]$ y encontrar el mínimo.

Complejidad: $O(n \cdot q)$



Un Segment Tree es una estructura de datos que permite responder consultas sobre rangos y actualizar elementos en $O(\log n)$.

Es un árbol binario donde cada nodo representa un segmento del arreglo.



El Segment Tree se construye de forma recursiva:

- La raíz representa todo el arreglo $[0, n - 1]$
- Cada nodo se divide en dos hijos: izquierdo $[l, m]$ y derecho $[m + 1, r]$
- Las hojas representan elementos individuales



Implementación

```
struct SegmentTree {
    int n;
    vector<int> tree;

    SegmentTree(vector<int>& a) {
        n = a.size();
        tree.resize(4 * n);
        build(a, 1, 0, n - 1);
    }

    void build(vector<int>& a, int node, int l, int r) {
        if (l == r) {
            tree[node] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(a, 2 * node, l, m);
        build(a, 2 * node + 1, m + 1, r);
        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
    }
};
```



Para responder una consulta [ql, qr]:

```
int query(int node, int l, int r, int ql, int qr) {  
    if (ql > r || qr < l) return INT_MAX;  
    if (ql <= l && r <= qr) return tree[node];  
    int m = (l + r) / 2;  
    int left = query(2 * node, l, m, ql, qr);  
    int right = query(2 * node + 1, m + 1, r, ql, qr);  
    return min(left, right);  
}
```



Update

Para actualizar un elemento en la posición pos con valor val:

```
void update(int node, int l, int r, int pos, int val) {  
    if (l == r) {  
        tree[node] = val;  
        return;  
    }  
    int m = (l + r) / 2;  
    if (pos <= m)  
        update(2 * node, l, m, pos, val);  
    else  
        update(2 * node + 1, m + 1, r, pos, val);  
    tree[node] = min(tree[2 * node], tree[2 * node + 1]);  
}
```


Complejidad

Operación	Complejidad
Construcción	$O(n)$
Query	$O(\log n)$
Update	$O(\log n)$
Memoria	$O(n)$

Segment Tree Lazy

Enzo Vivallo

enzovivallo@estudiante.uc.cl



Problema

Se te entrega una lista de enteros a de tamaño n ($1 \leq n \leq 2 \cdot 10^5$), y cada elemento cumple $|a_i| \leq 10^9$. Nos piden responder q ($1 \leq q \leq 2 \cdot 10^5$) consultas del tipo:

- ¿Cuál es el mínimo en el rango $[l, r]$?
- Sumar un valor v a todos los elementos en el rango $[l, r]$



Lazy Propagation

Cuando hacemos update en un rango, actualizar todos los nodos afectados sería $O(n \log n)$ en el peor caso.

La técnica de **Lazy Propagation** pospone las actualizaciones: solo actualizamos cuando es necesario.



- Cuando una actualización cubre completamente un nodo, marcamos ese nodo como "pendiente" (lazy) y no propagamos a sus hijos
- Solo cuando necesitamos acceder a los hijos, propagamos el valor pendiente
- Esto mantiene la complejidad $O(\log n)$ por operación



Implementación

```
struct SegmentTreeLazy {
    int n;
    vector<int> tree, lazy;

    SegmentTreeLazy(vector<int>& a) {
        n = a.size();
        tree.resize(4 * n);
        lazy.resize(4 * n, 0);
        build(a, 1, 0, n - 1);
    }

    void build(vector<int>& a, int node, int l, int r) {
        if (l == r) {
            tree[node] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(a, 2 * node, l, m);
        build(a, 2 * node + 1, m + 1, r);
        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
    }
};
```



Propagación

```
void propagate(int node, int l, int r) {  
    if (lazy[node] != 0) {  
        tree[node] += lazy[node];  
        if (l != r) {  
            lazy[2 * node] += lazy[node];  
            lazy[2 * node + 1] += lazy[node];  
        }  
        lazy[node] = 0;  
    }  
}
```



Update con Lazy

```
void update(int node, int l, int r, int ql, int qr, int val) {  
    propagate(node, l, r);  
    if (ql > r || qr < l) return;  
    if (ql <= l && r <= qr) {  
        lazy[node] += val;  
        propagate(node, l, r);  
        return;  
    }  
    int m = (l + r) / 2;  
    update(2 * node, l, m, ql, qr, val);  
    update(2 * node + 1, m + 1, r, ql, qr, val);  
    tree[node] = min(tree[2 * node], tree[2 * node + 1]);  
}
```



Query con Lazy

```
int query(int node, int l, int r, int ql, int qr) {  
    propagate(node, l, r);  
    if (ql > r || qr < l) return INT_MAX;  
    if (ql <= l && r <= qr) return tree[node];  
    int m = (l + r) / 2;  
    int left = query(2 * node, l, m, ql, qr);  
    int right = query(2 * node + 1, m + 1, r, ql, qr);  
    return min(left, right);  
}
```




Operación	Complejidad
Construcción	$O(n)$
Query	$O(\log n)$
Range Update	$O(\log n)$
Memoria	$O(n)$